

Sorting Algorithms – Data Structures

Vlad-Robert Baci

March 2026

1 Introduction

There are 2 hard problems in computer science: cache invalidation, naming things, and off-by-1 errors. – Leon Bambrick

Sorting might actually be one of the hard problems in computer science. Or at least it was in the 1900s.

We all, excellent programmers, know better: if there's a convenient sorting function we can call in our programming language, be assured that we will not reinvent the wheel... But why not reinvent the wheel? Even worse, what if the environment we work in doesn't provide us with the implementation of a mystical black box that will sort our data?

In this case study, I intend to compare the most widely known sorting algorithms and see how they perform on different datasets. Is it even possible to write an algorithm that gets close to the performance of Intro sort, the algorithm used by `std::sort`, without the help of twenty experienced Japanese engineers?

2 Algorithm selection

I have decided to benchmark these major and well known sorting algorithms:

Algorithm	Time Complexity	Space
Bubble sort	$O(n^2)$	$O(1)$
Insertion sort	$O(n^2)$	$O(1)$
Selection sort	$O(n^2)$	$O(1)$
Merge sort	$O(n \log n)$	$O(n)$
Heap sort	$O(n \log n)$	$O(1)$
Quick sort	$O(n \log n)$	$O(\log n)$
Intro sort	$O(n \log n)$	$O(\log n)$

Bubble, insertion and selection sort have a suboptimal time complexity, so we expect them to perform worse than merge, heap and quick sort, which have an optimal time complexity. Even if it might be tempting to decline the first three algorithms of any appreciation, one undisputed advantage lies in how simple they are to implement.

Note: I did not implement Intro sort, instead, I plotted the `std::sort` implementation for reference.

3 Datasets

To properly benchmark these algorithms, the datasets must be varied and mimic real-world data. I developed four fill strategies:

Fill	Description
Random	Random elements within a specified range.
Sorted	Random elements sorted ascending.
Reverse-sorted	Random elements sorted descending.
Constant	All elements equal to zero.

The range (density) of the random elements is as follows:

Category	Range	Description
Wide	$[0, N]$	Few duplicates.
Dense	$[0, N/32]$	Frequent duplicates.
Logarithmic	$[0, \log_2 N]$	Extremely frequent duplicates.

Finally, I tested across three data types:

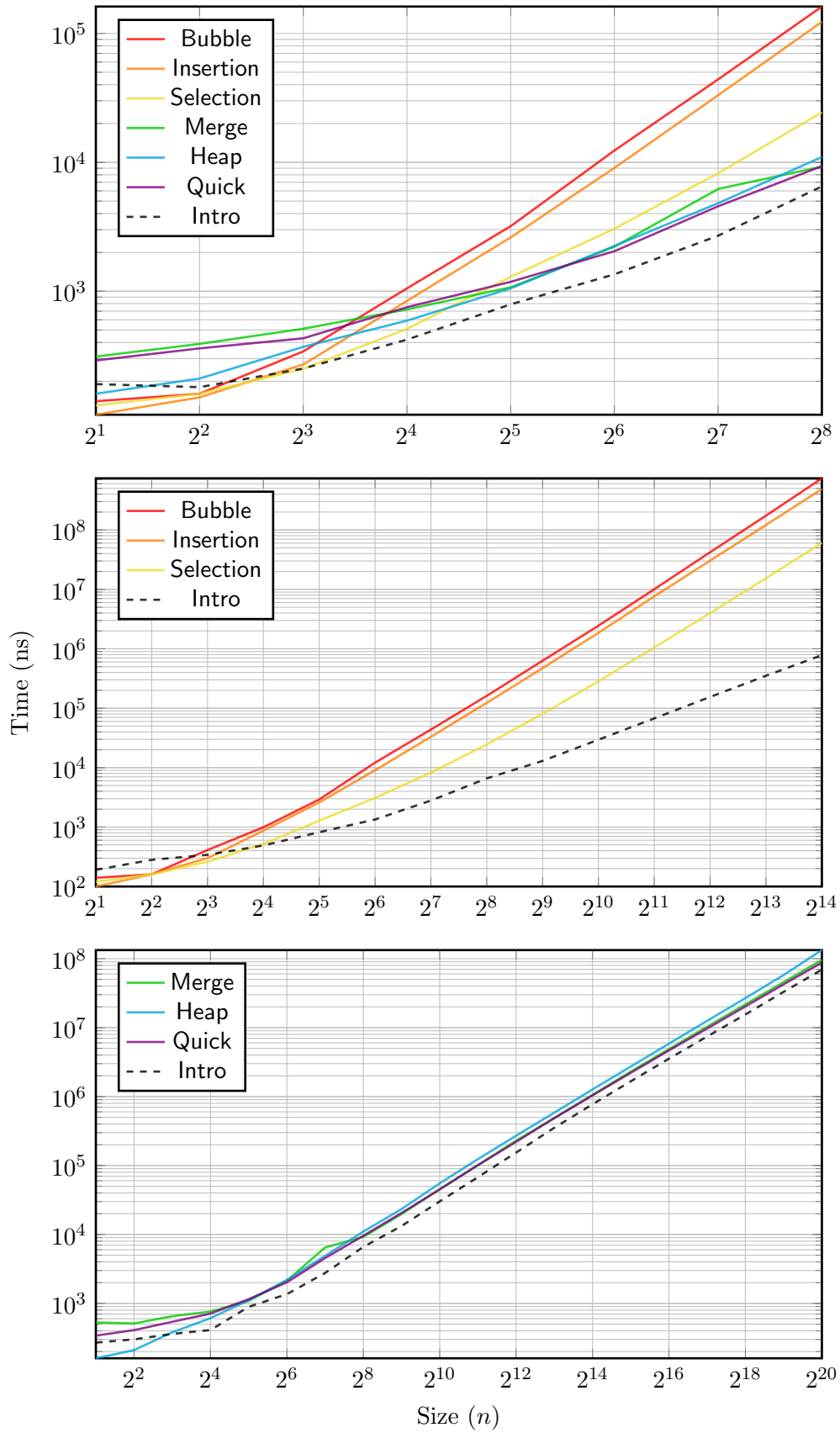
Data type	Impact
<code>byte</code>	Narrower data type; expecting improved performance due to less cache misses.
<code>int</code>	Standard choice.
<code>double</code>	Wider data type; expecting worsened performance due to FPU/SIMD comparisons overhead.

4 Environment

C++ was the definitive choice, because of its performance and powerful templates. The source code was compiled with GCC 15 and the `-O3` optimization flag.

5 Initial benchmarking

Random fill, wide density, int data



For $n \leq 8$, Bubble, Insertion and Selection sort perform better than Merge, Heap or Quick, because the latter have a bigger initial overhead. Merge sort can be seen as the worst performing algorithm for very small n .

For $n \geq 32$, $O(n \log n)$ algorithms start to outperform the previously winning $O(n^2)$ sorts.

At $n = 128$, Bubble and Insertion sort are already 10 times slower, but Selection sort is only 2 times slower than all $O(n \log n)$ algorithms.

As n grows huge, Merge, Heap and Quick sort all grow at the same rate, not far away from Intro sort. Quick sort wins over the other two, but not by much. Bubble and Insertion sort become almost 1000 times slower, close to seconds in magnitude, yet Selection sort is "only" 100 times slower.

Selection sort is the winner of $O(n^2)$ algorithms, yielding the best overall performance for small n , while also being significantly better than Bubble or Insertion sort as n grows exponentially. A simple algorithm to implement without any pain, in any programming language, excluding x86 Assembly.

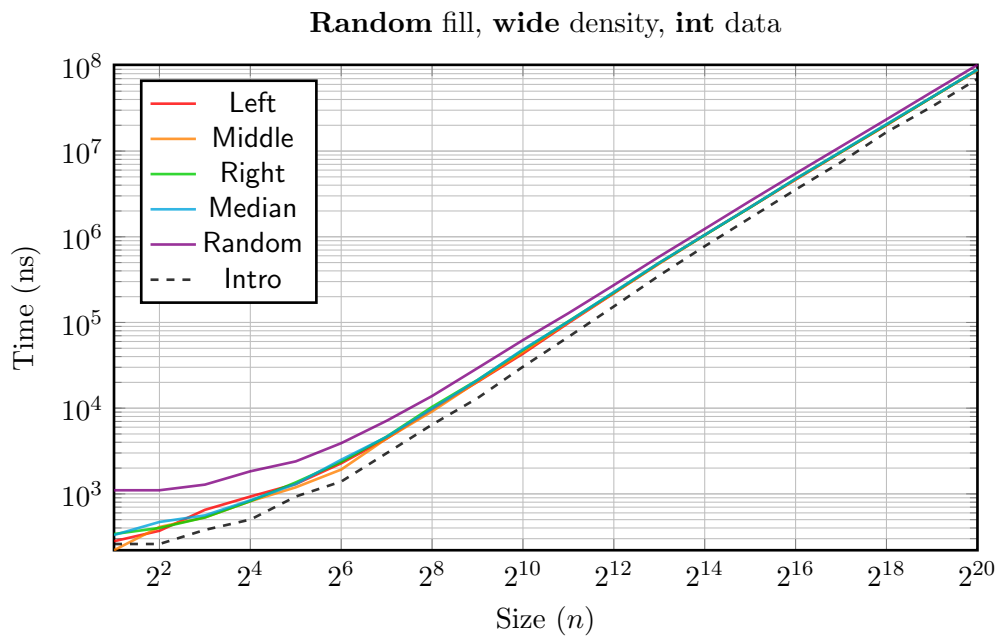
For $O(n \log n)$ algorithms, I would declare Quick sort the winner. Even if Heap sort is somewhat better for very small n , Quick sort wins for large n .

If your dataset is small- or medium-sized and you don't need that cutting edge performance, implement **Selection sort**. However, if your dataset is very large, you should go with **Quick sort**.

6 Quick sort – Pivot selection

The performance of Quick sort is strongly linked to the *pivot* selection strategy. For the previous benchmark, the middle element was silently chosen, but several other strategies exist:

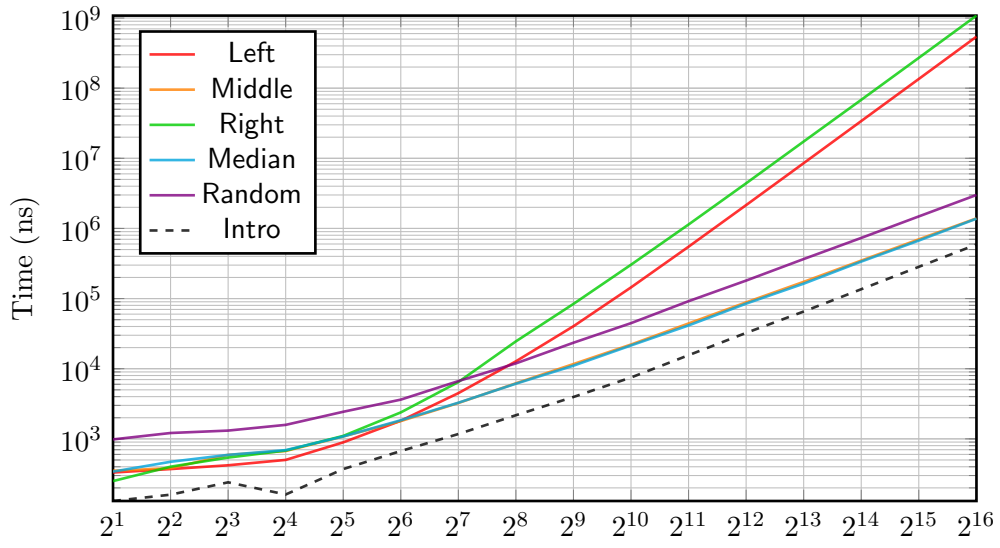
- left-most element
- middle element (used previously)
- right-most element
- median-of-three
- random element



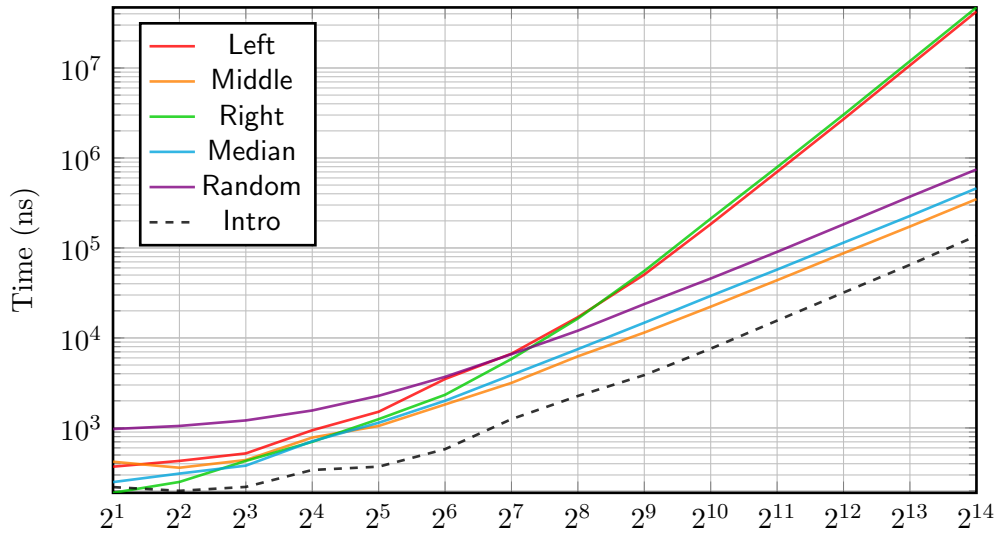
In this benchmark, all *pivot* selection strategies are very close to one another in terms of performance, with the random strategy yielding the worst speed, because of the `rand()` function overhead.

It might seem that one is free to choose from the other four strategies freely, but Quick sort has a worst case time complexity of $O(n^2)$ so the *pivot* selection is only insignificant for a random dataset!

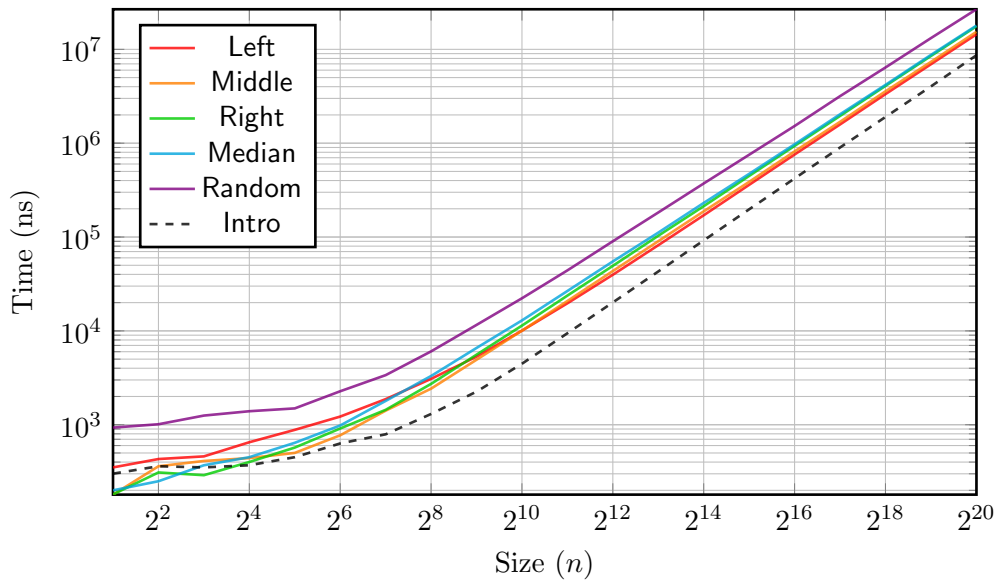
Sorted fill, wide density, int data



Reverse-sorted fill, wide density, int data



Constant fill, int data



On sorted or reverse-sorted datasets, the Left and Right strategies are absolutely disastrous, showing how bad Quick sort can perform if the *pivot* is chosen poorly. For $n \geq 2^{18}$, the huge recursion depth triggered a stack overflow and subsequently, the process segfaulted. These strategies were outperformed even by a Random *pivot*.

Median-of-three and Middle are equally good for sorted datasets, but Middle wins in a reverse-sorted benchmark.

For constant datasets, all strategies perform, for all practical reasons, the same. The Left strategy seems to be somewhat faster compared to the rest.

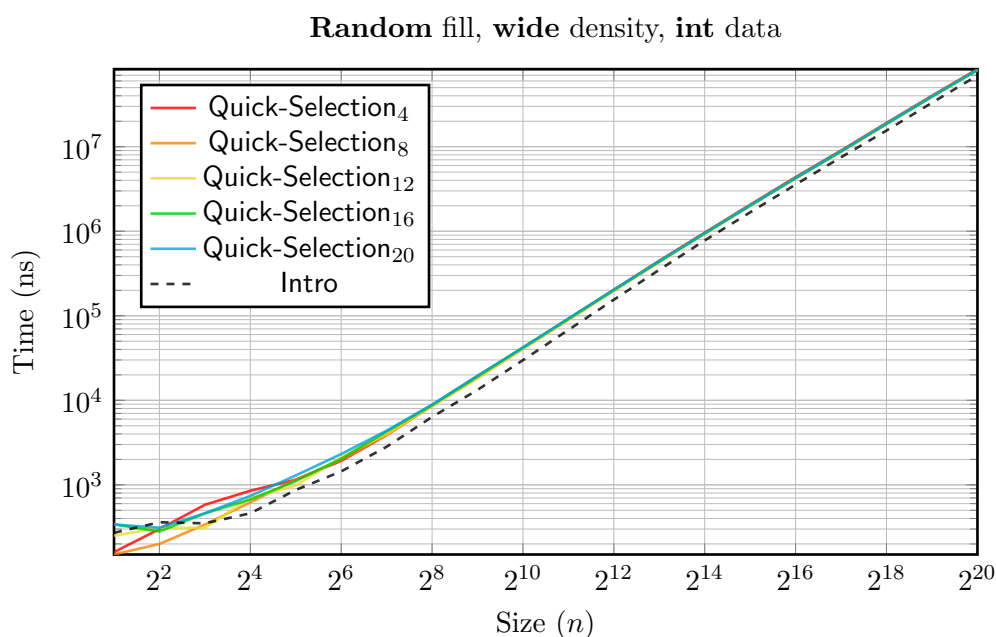
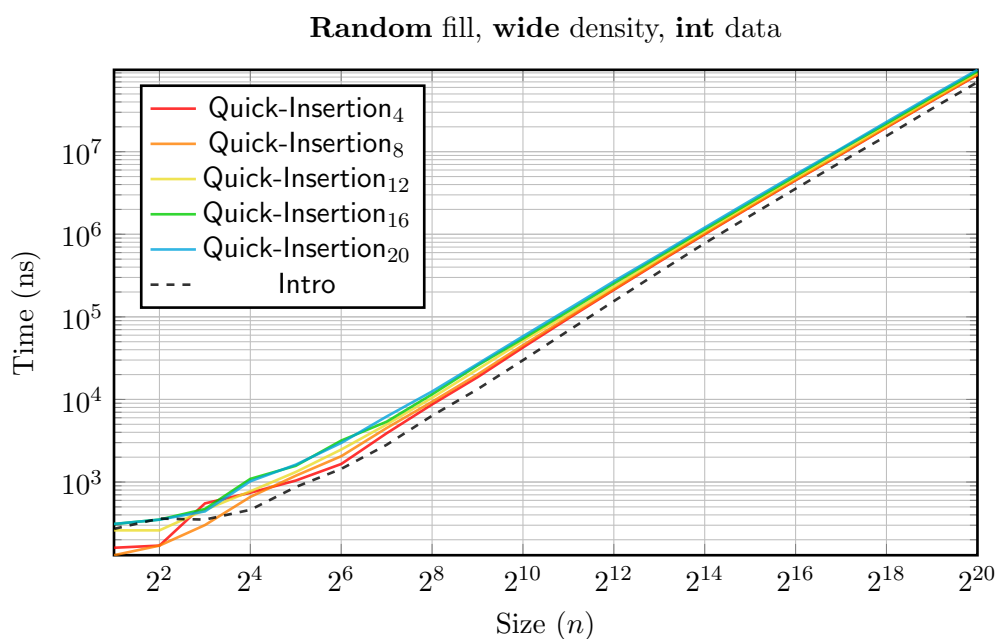
Middle is the winning strategy for Quick sort. This is the strategy that will be used throughout this paper, when referring Quick sort.

7 Hybrid sort

As shown in Section 5, Merge, Heap and Quick sort are not the most efficient algorithms for small n . A hybrid algorithm could be developed, one that uses an $O(n \log n)$ algorithm as usual, but when n gets small, it switches to a simpler sort.

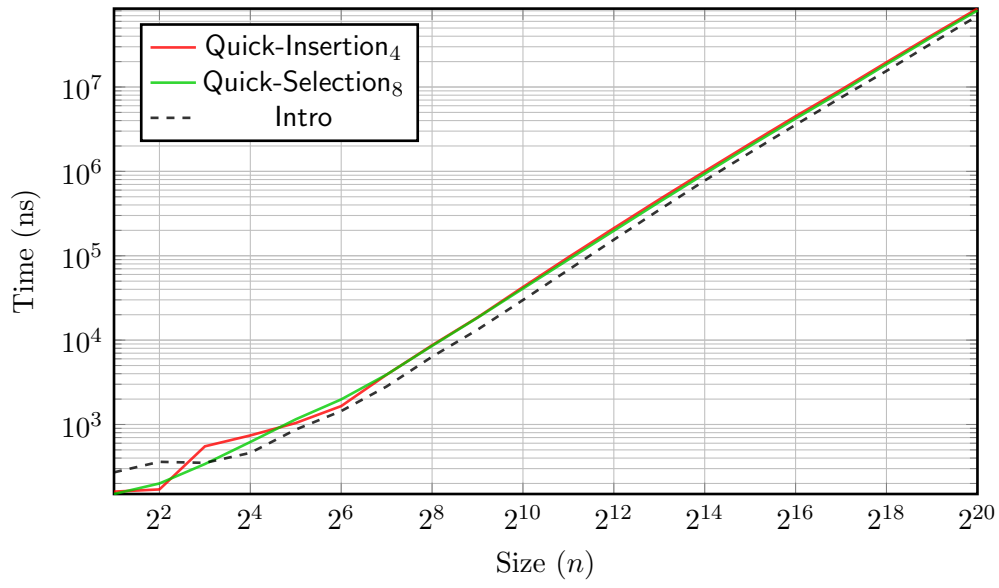
Since Quick sort was the fastest for large datasets, I only needed to choose the algorithm for small n . Bubble sort was the worst one out of all $O(n^2)$ sorts, so I had to choose from Insertion or Selection sort.

Beside the algorithm itself, the **threshold** at which the algorithm quits using Quick sort impacts performance. I benchmarked several hybrid algorithms that pair Quick sort with Insertion or Selection sort at different thresholds: $\{4, 8, 12, 16, 20\}$.



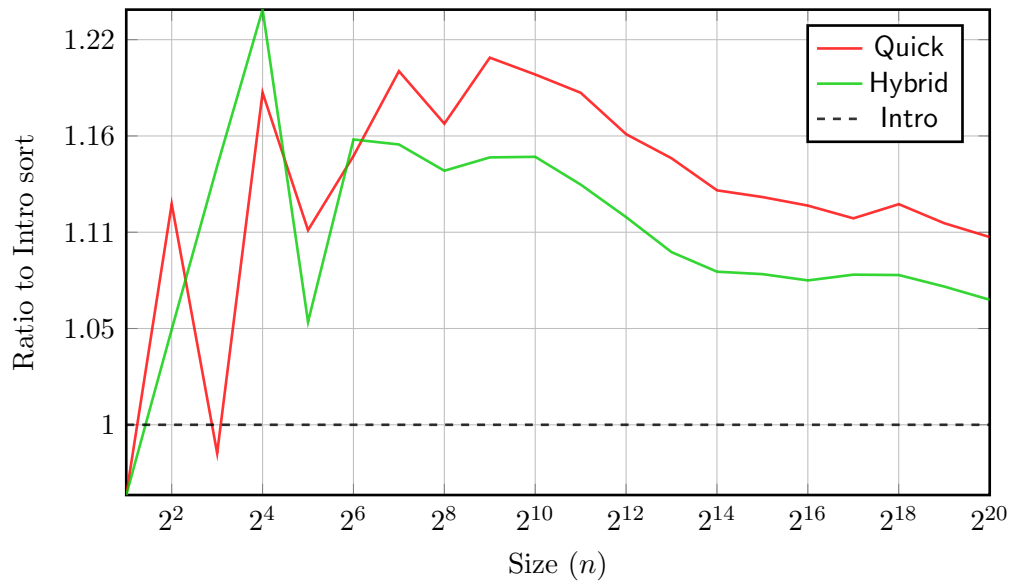
Quick-Insertion₄ and Quick-Selection₈ seemed the most promising, so I put them up for a legendary battle!

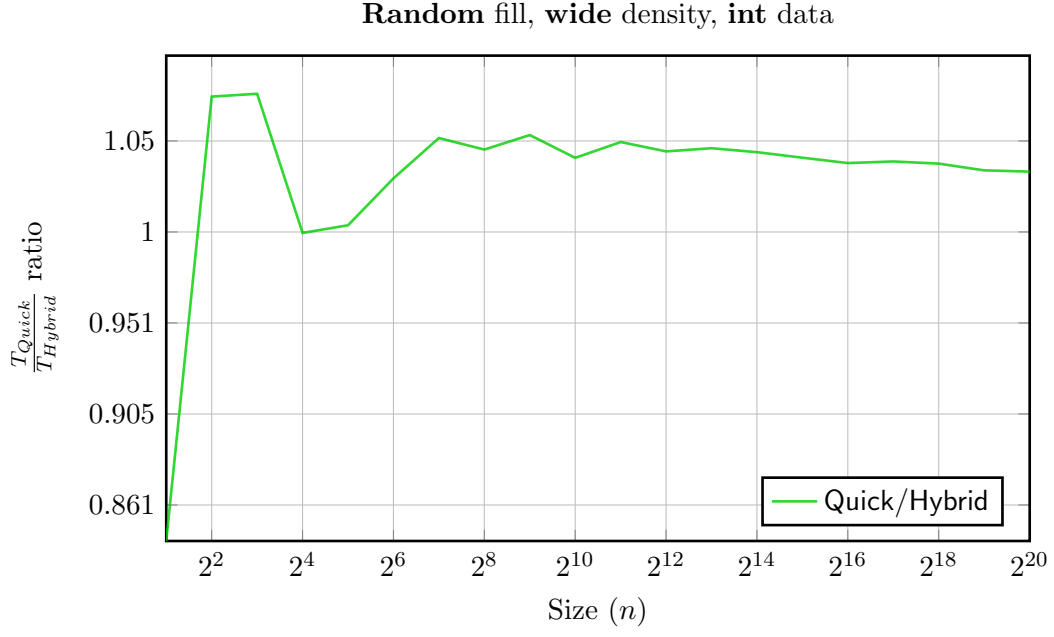
Random fill, wide density, int data



The best hybrid is Quick-Selection₈: apply Quick sort if $N > 8$ and Selection sort otherwise.

Random fill, wide density, int data



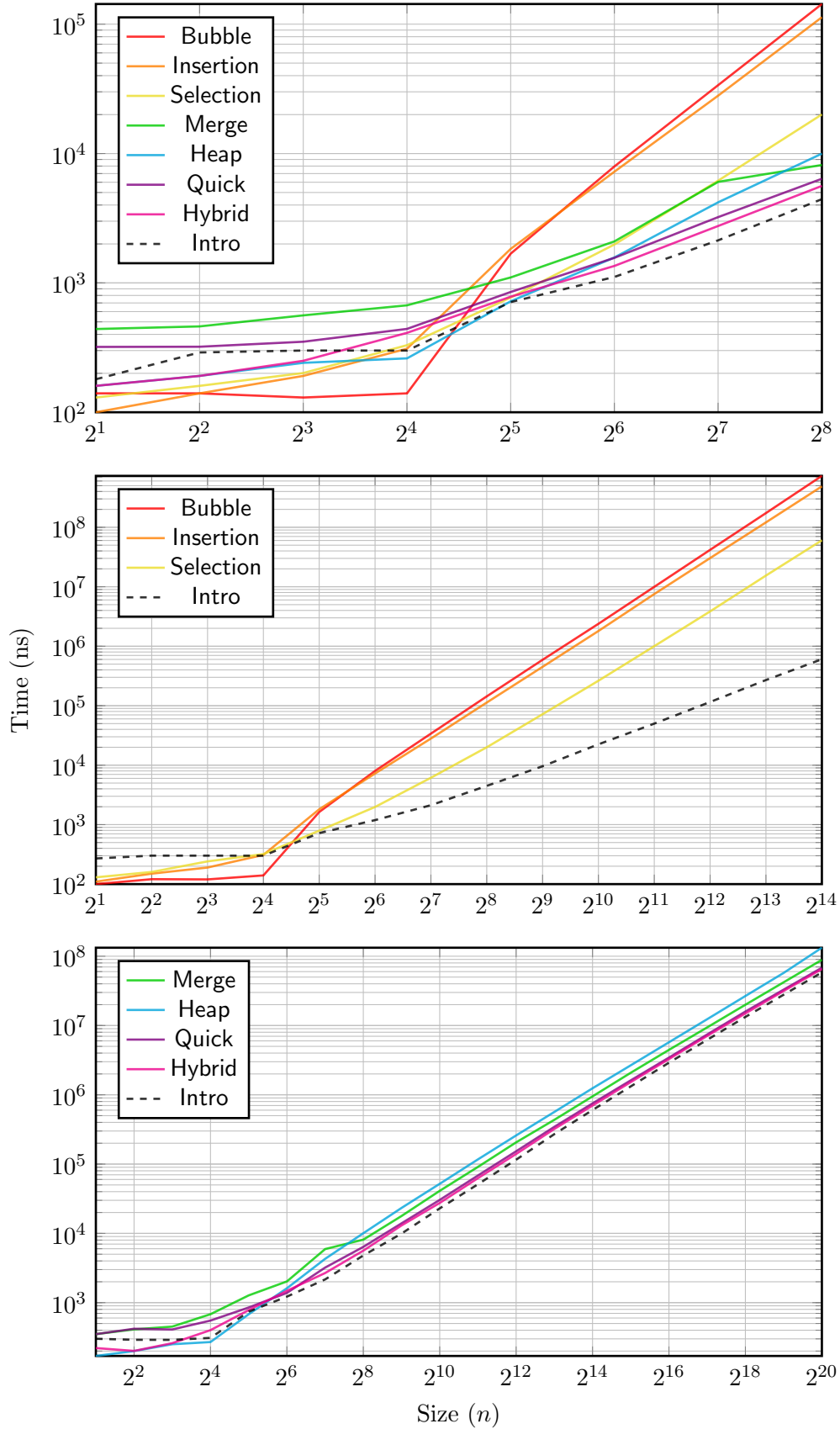


Quick-Selection₈ is the algorithm that will be used throughout this paper when referring to Hybrid sort.

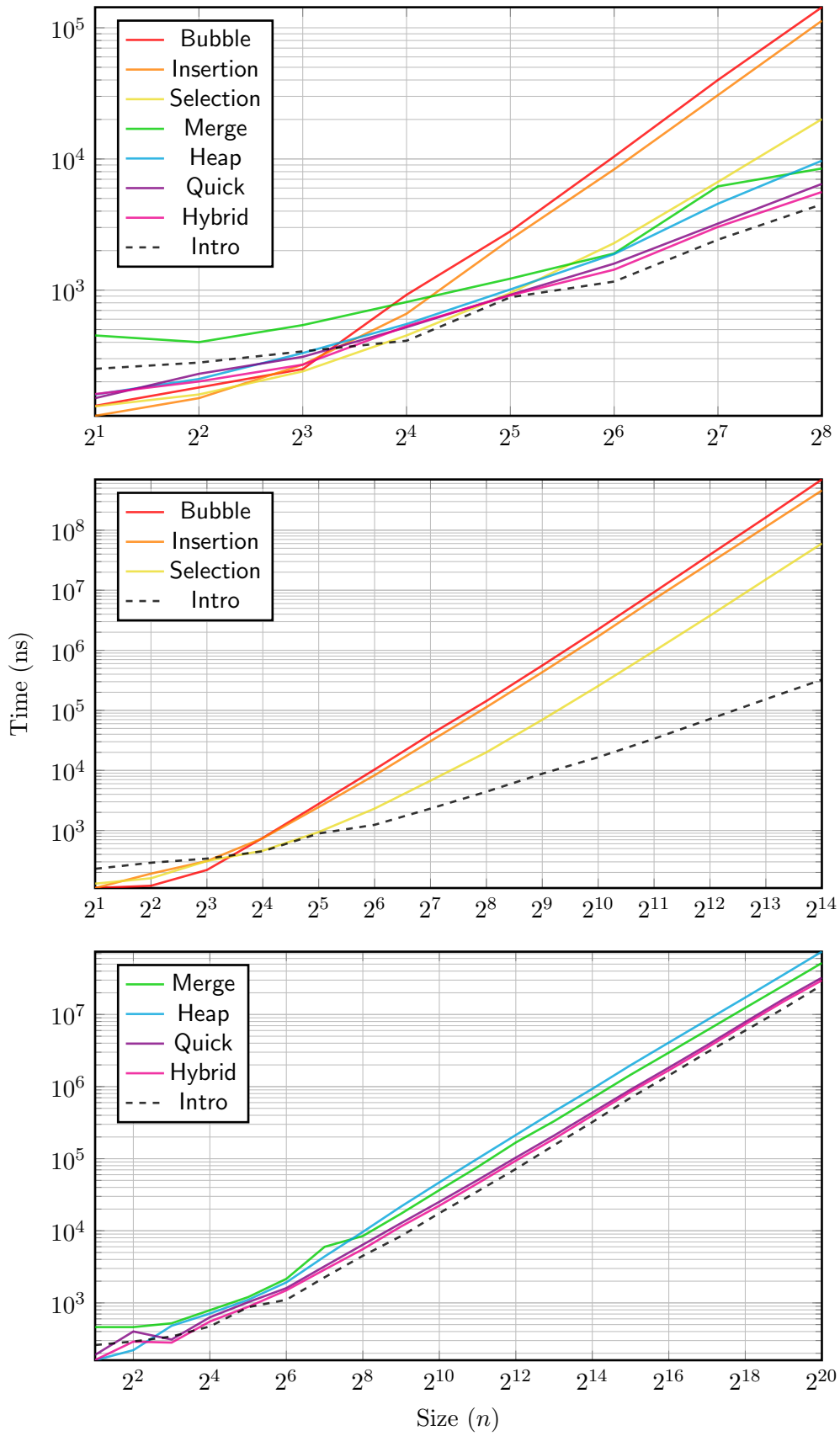
The plotted line is above $y = 1$, so our hybrid algorithm is in fact better than a plain Quick sort. It's around 3 – 5% faster, when benchmarking against a random dataset.

8 More benchmarking

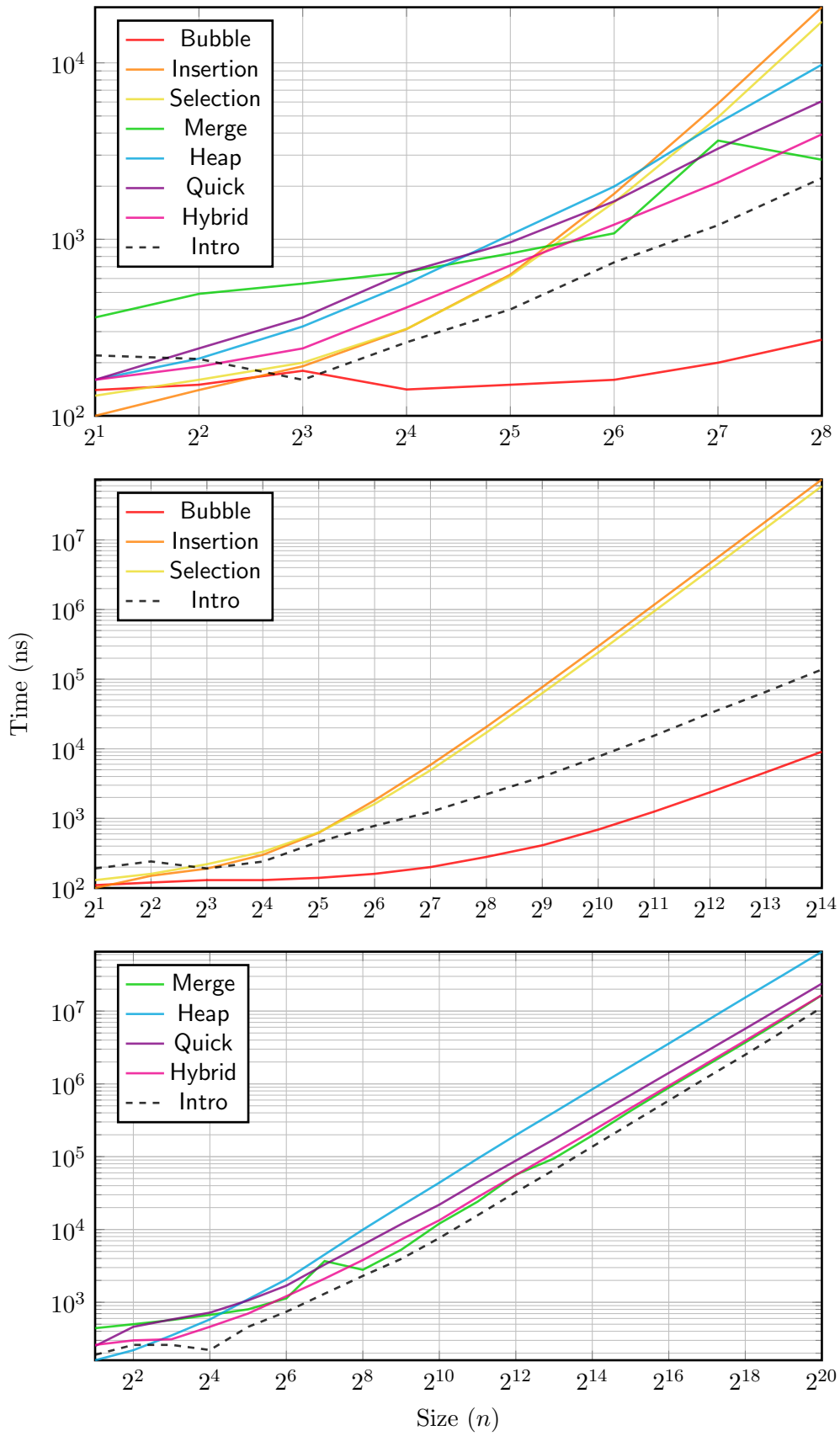
Random fill, dense density, int data



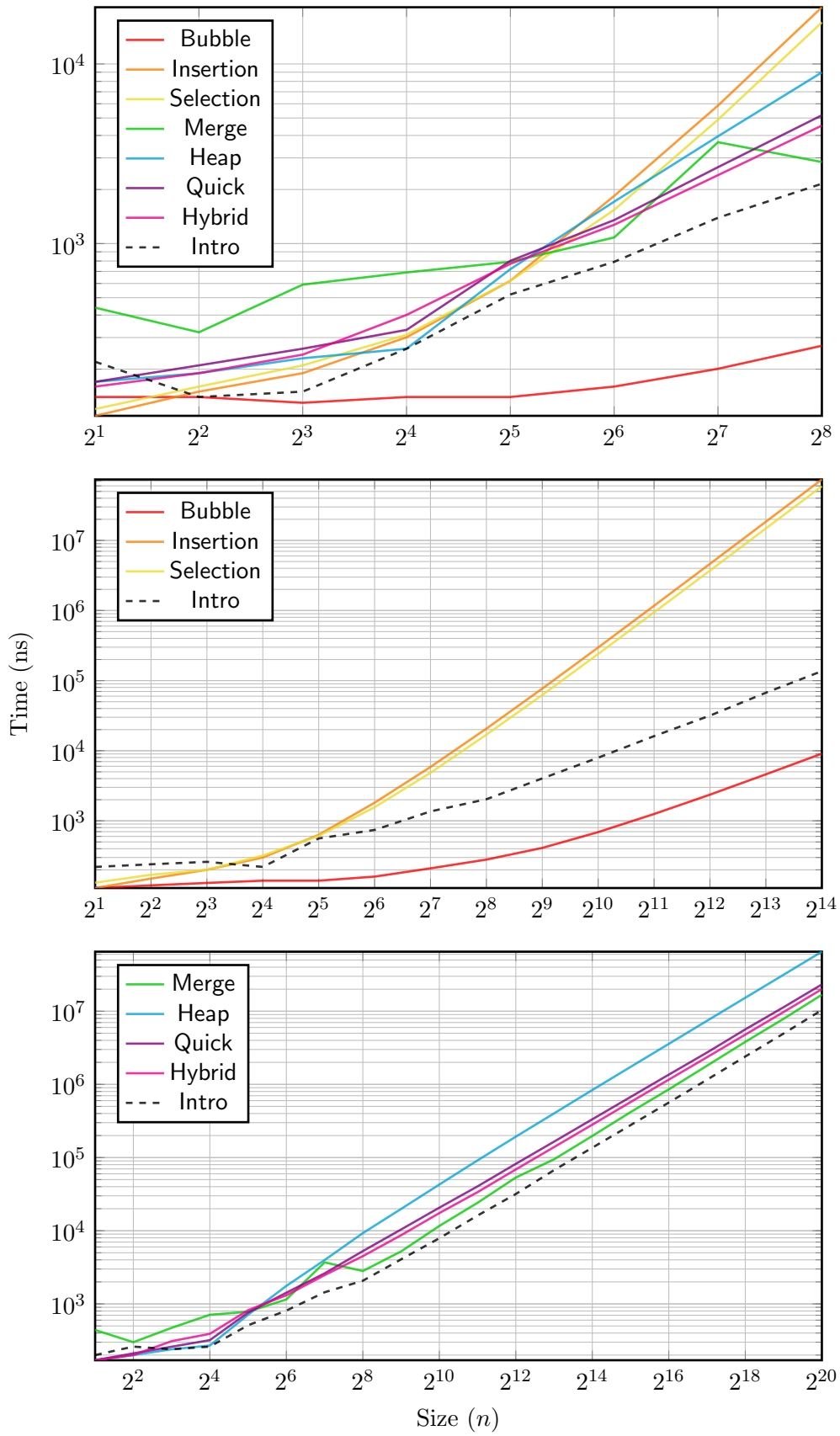
Random fill, logarithmic density, **int** data



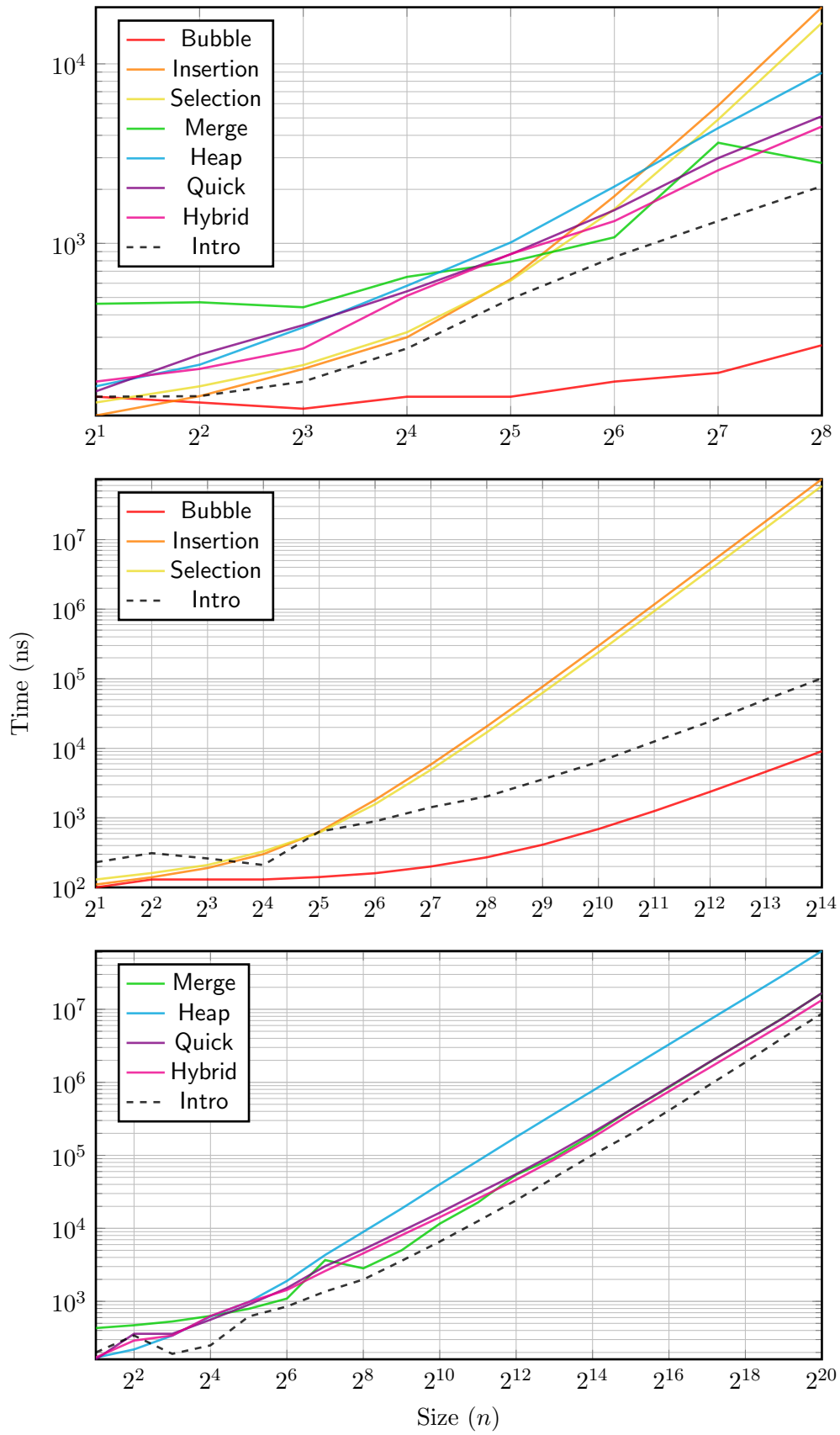
Sorted fill, wide density, int data



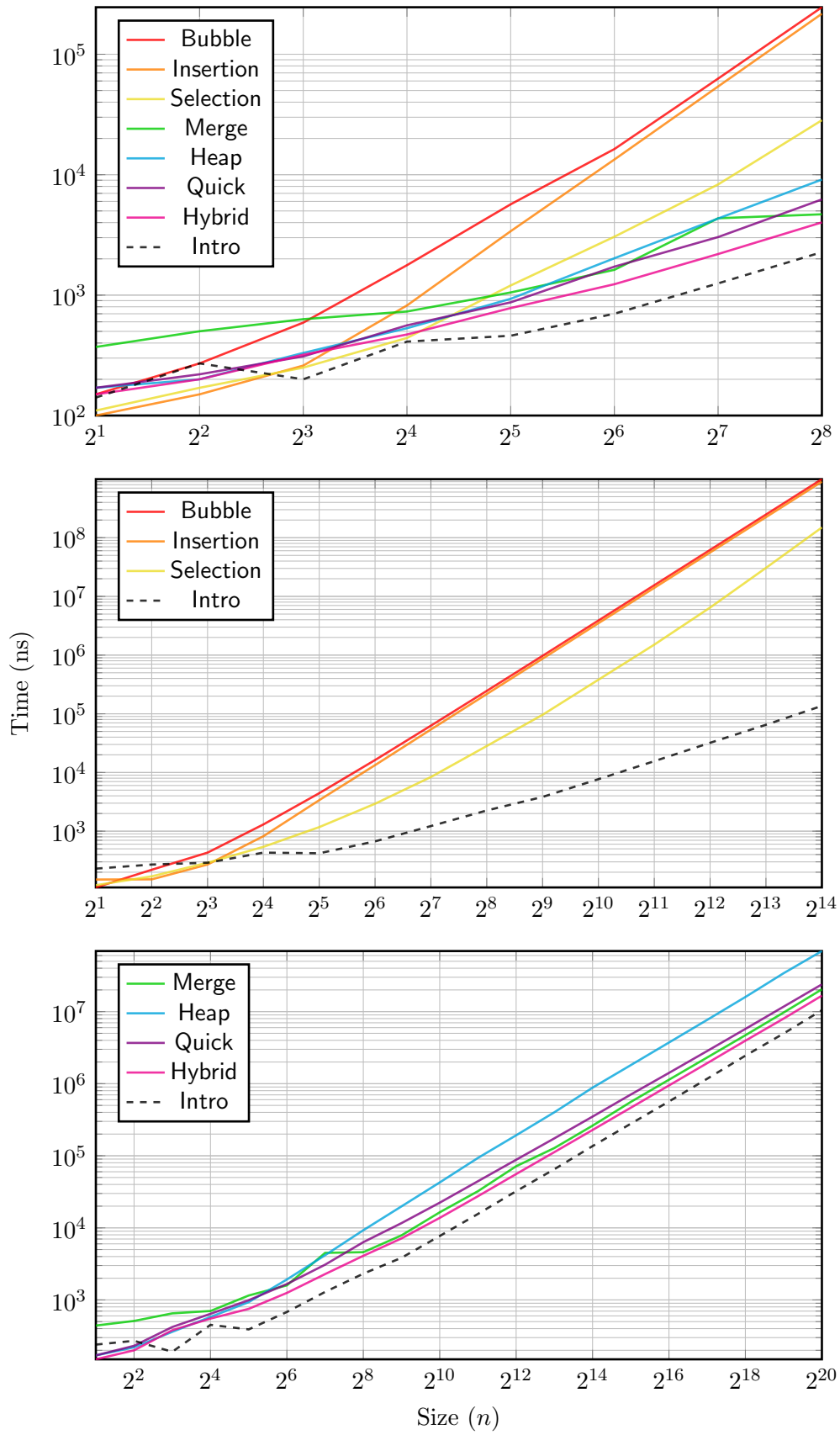
Sorted fill, dense density, **int** data



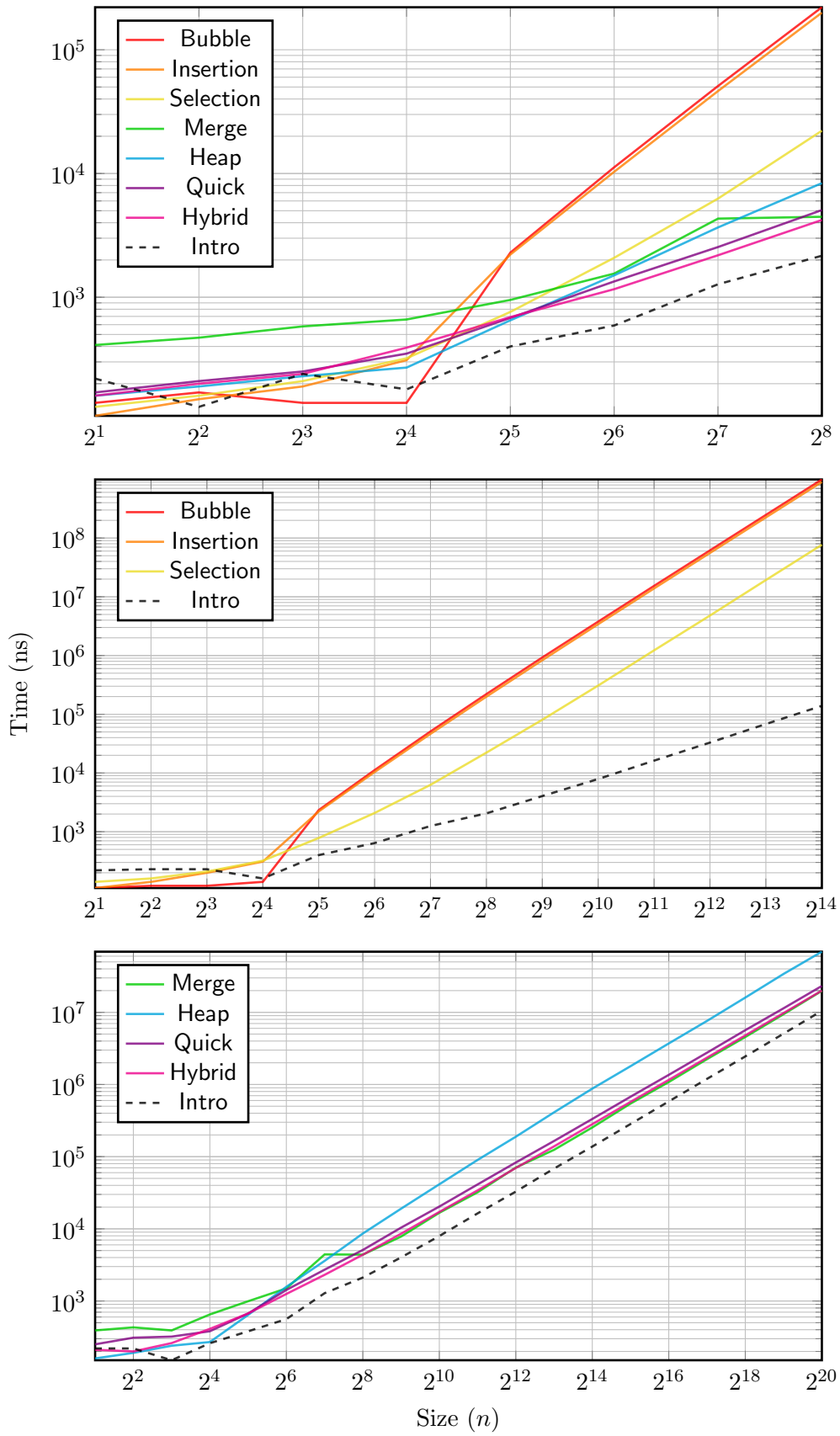
Sorted fill, logarithmic density, int data



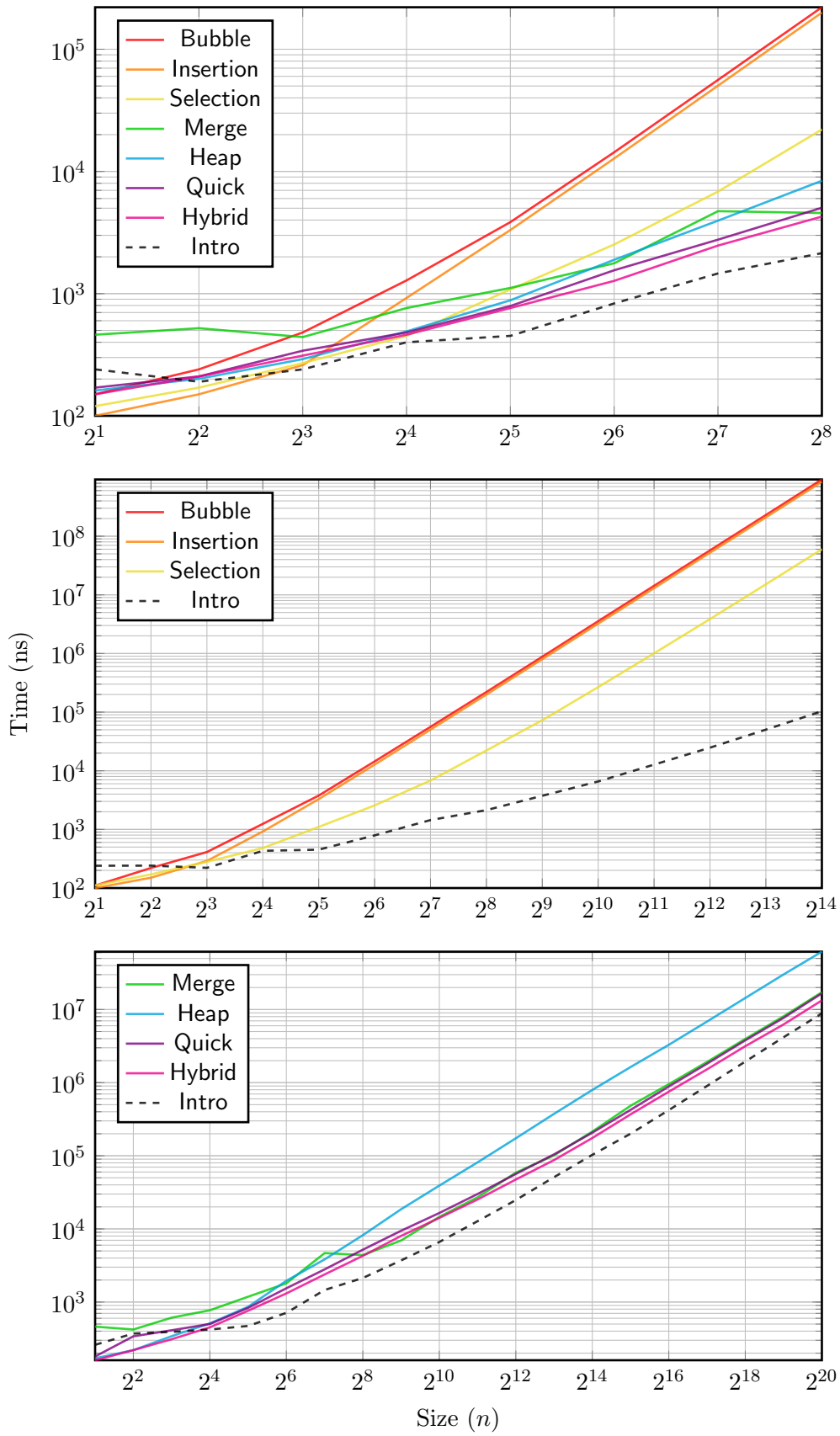
Reverse-sorted fill, wide density, int data



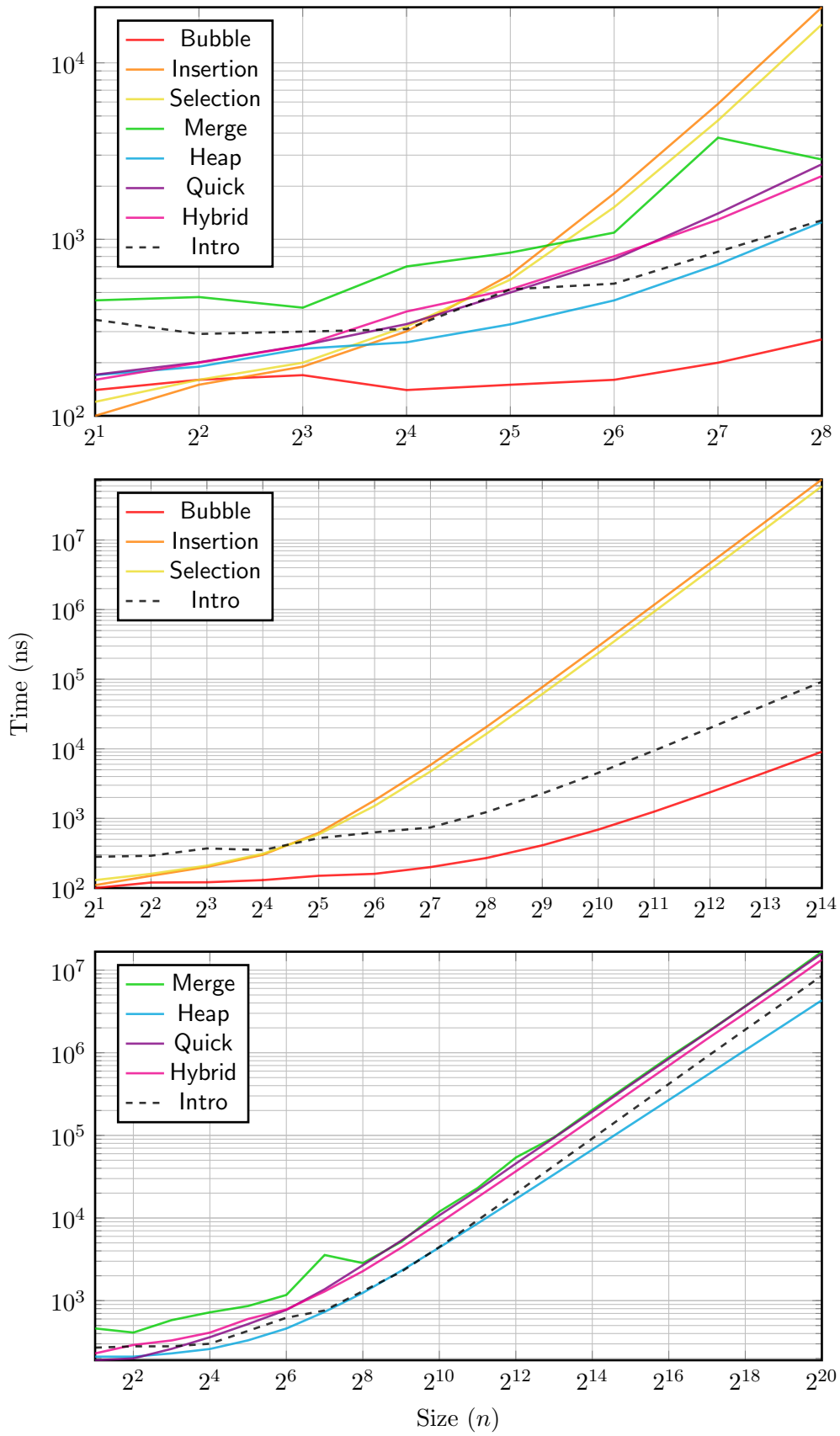
Reverse-sorted fill, dense density, int data



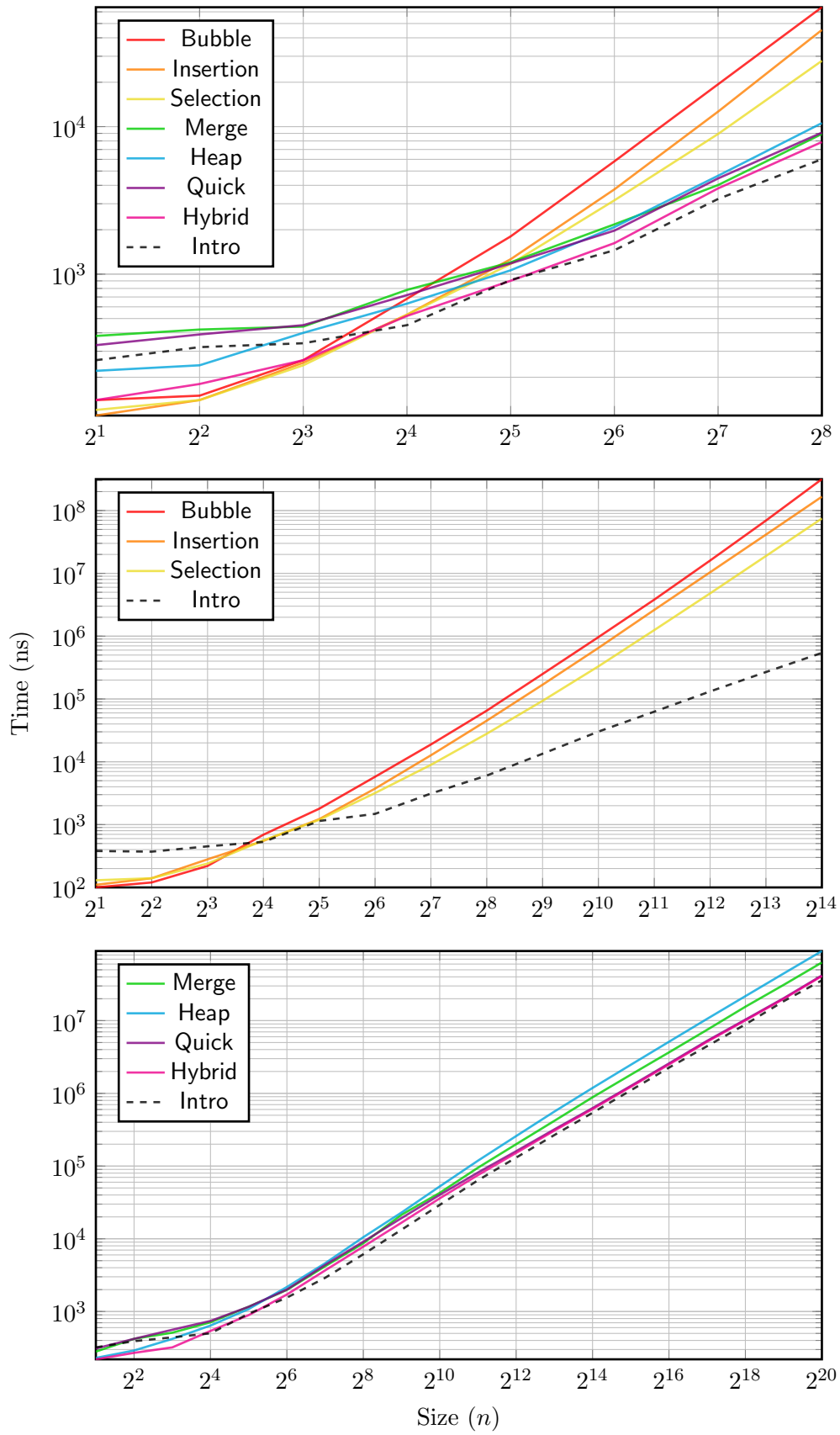
Reverse-sorted fill, log density, int data



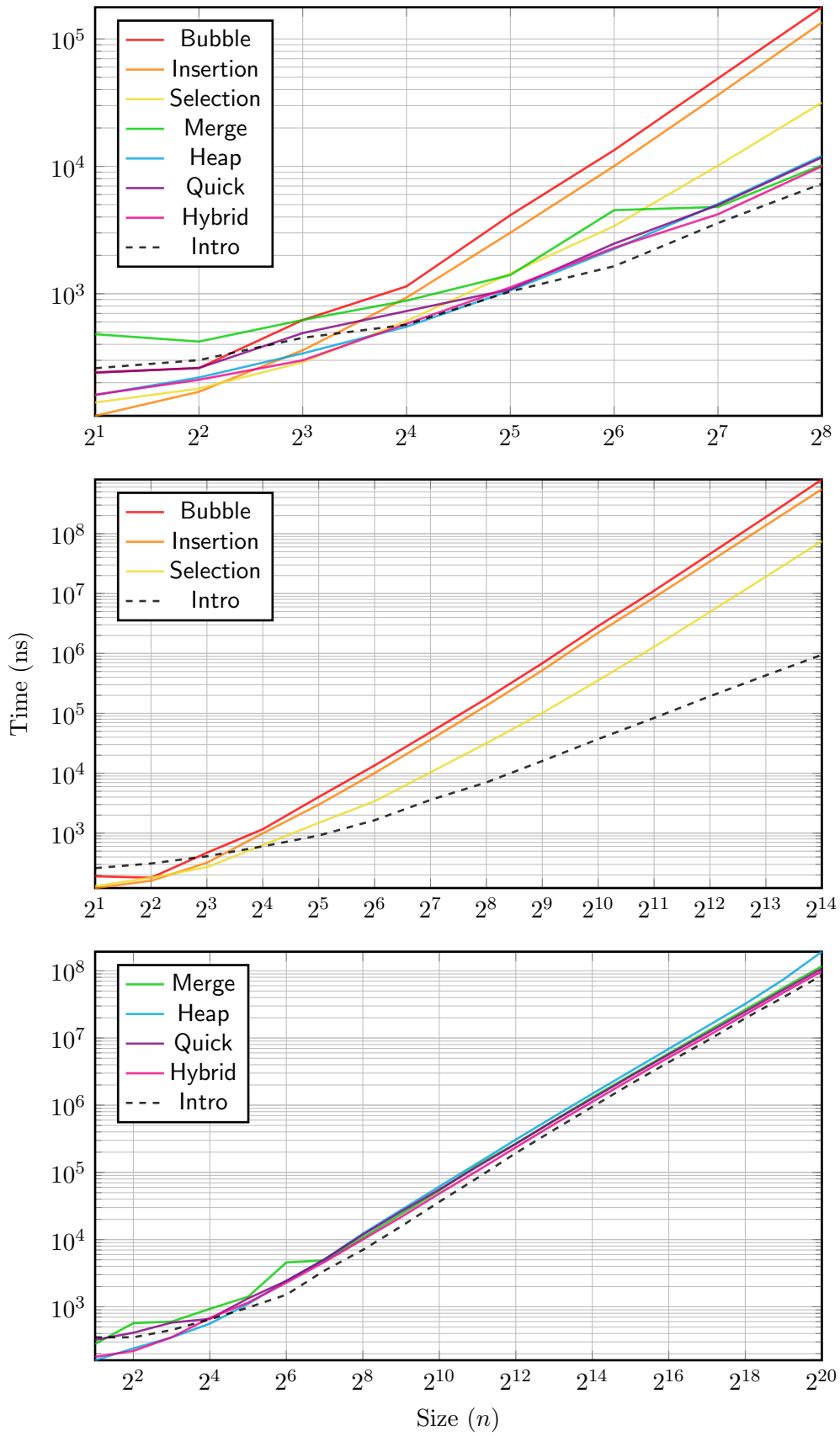
Constant fill, int data



Random fill, wide density, byte data



Random fill, wide density, double data



For a sorted fill, Bubble sort has the best performance, outperforming all algorithms, including Intro sort. Insertion improves, so it gets closer to Selection sort it previously was. If the density is wide or dense, Hybrid sort is also a bit slower than Merge sort, but it always wins over Quick sort.

If the fill is reverse-sorted, unfortunately Bubble and Insertion behave equally bad, and if the density is wide, the performance of Selection sort also degrades. Heap sort is the worst out of all $O(n \log n)$ sorts, followed by Quick sort. Merge sort and Hybrid sort battle, with Hybrid winning on wide and logarithmic densities, but losing on dense.

When the dataset is constant, Bubble sort beats the others again. Heap sort outperforms Hybrid and Intro sort in this case. Hybrid gets 4th place, after Bubble, Heap and Intro sort.

Some remarks can be made when testing on other primitives. If sorting **bytes**, both $O(n^2)$ and $O(n \log n)$ algorithms tend to group together faster. Selection sort performs almost as bad as Bubble and Insertion sort. As n grows, Hybrid sort gets very, very close to the performance of Intro sort. For the **double** data type, all $O(n \log n)$ are very close to Intro sort in terms of performance.

All of the six algorithms that I have benchmarked have proved themselves useful in some way. Bubble performs best on sorted or constant datasets. Selection sort is a nice overall choice for small- or medium-sized datasets. Merge is nice because of its simplicity, while being of optimal time complexity. Heap sort is interesting because of its rather unique implementation idea. Quick sort is, well, very quick in general, and if you are careful with the *pivot* choice, the performance is notable. The most uninteresting one could be Insertion sort, but it's still a simple and interesting algorithm.

I might not have beaten Intro sort, as implemented in `std::sort`, but I got pretty close with my hybrid algorithm. It turns out you don't need twenty Japanese engineers for such performance.

9 Benchmarking accuracy

- The recorded sorting time was the minimum of 25 runs
- Before each benchmark, the sort was ran 3 times to prime the system
- No other unnecessary apps or programs were running when benchmarking

10 To be continued...

This study is far from complete. As with any problem that arises in engineering, there is no simple answer, and the choice depends on many factors.

I only took into account the execution time, but I didn't consider the number of comparisons and swaps for each algorithm. If every swap matters, you might want to implement Selection sort. If you just want the speed, go on and use Quick sort. Does your dataset have a huge chance of being sorted? Use Bubble sort. If you are lazy, use Bubble sort.

Space critical you say? Don't you think about Merge sort. You can also try and implement a Hybrid sort if you want to. Me? I'll ride with Bogosort.

In the end, the choice is yours. Don't overthink it, and don't be scared of choosing.

Turns out this wasn't about sorting algorithms.